

SERVIÇO NACIONAL DE APRENDIZAGEM INDUSTRIAL – SENAI
CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMA

ALICE VIRGÍLIA ANDRADE

DOCUMENTAÇÃO INDIVIDUAL SOBRE OS TESTES UNITÁRIOS
Iveco Green Ledger - Sistema de Rastreamento Inteligente

NOVA LIMA - MG
2026

ALICE VIRGÍLIA ANDRADE

DOCUMENTAÇÃO INDIVIDUAL SOBRE OS TESTES UNITÁRIOS

Iveco Green Ledger - Sistema de Rastreamento Inteligente

Documentação de testes unitários apresentado ao Curso de Desenvolvimento de Sistemas do Serviço Nacional de Aprendizagem Industrial (SENAI), como requisito para a avaliação da Situação de Aprendizagem do projeto de Trabalho de Conclusão de Curso (TCC)

Orientador: Frederico Martins Aguiar

NOVA LIMA - MG

2026

1 - IDENTIFICAÇÃO DO PROJETO

Nome: Alice Virgília Andrade (<https://github.com/aliceandradee>).

Turma: Desenvolvimento de Sistemas.

Nome da Equipe: Green Ledger (https://github.com/NicolasOlim/Ivec_Green_Ledger.git).

Data de Entrega: 25/08/2026.

Componente escolhido: DadosServiceTestes, EmailValidationServiceTestes, IEmailValidationServiceTestes.

Métodos e Serviços Testados: CriarLoteMateriaPrima(), CriarVeículoComponente(), ExcluirFornecedor(), ExcluirVeículo(), ValidateEmailAsync().

1.1 Justificativa da Escolha

A camada **Services** contém as regras de validação de negócios, tratando dados antes que sejam gravados no banco de dados ou integrados a outras dependências externas. A escolha dessa camada busca garantir que exceções temporais, validações de domínio restrito e verificações de limites numéricos funcionem corretamente de forma isolada.

2 RESPONSABILIDADE DO COMPONENTE

Os serviços testados são responsáveis por validar a consistência e a integridade das informações enviadas à API antes de qualquer operação de persistência ou consulta.

- **DadosService:** Gerencia as regras de negócio para criação de lotes de matéria-prima, cadastro de componentes de veículos e exclusão de fornecedores e veículos, garantindo que nenhum dado inconsistente chegue à camada de dados.
- **EmailValidationService:** Valida a autenticidade e o formato dos e-mails de usuários corporativos, aplicando higienização de strings (Trim), verificação de caixa alta/baixa.

3 ESTRATÉGIA DE TESTES E COMPORTAMENTOS MAPEADOS

A estratégia adotada utilizou objetos simulados (*mocks*) via biblioteca Moq para isolar o componente de log (ILogger<DadosService>) e o cache em memória (IMemoryCache). Para garantir que os testes avaliem exclusivamente as regras de validação, a instância do banco de dados (FireBaseData) foi definida como null nos cenários de teste.

- **Cenários Válidos:** Aceitação de e-mails do domínio, mesmo com espaços antes/depois ou letras maiúsculas ("**ADMIN@IVECO.COM**").
- **Cenários Inválidos:** Rejeição de e-mails de domínios genéricos (gmail.com, yahoo.com), tentativas de burla de subdomínio (iveco.com.br), strings nulas ou em branco, IDs de fornecedor vazios e VINs nulos.
- **Fora do Escopo:** Persistência de dados real no Firebase e comunicação de rede HTTP de ponta a ponta.

4 TESTES DESENVOLVIDAS

Os testes foram executados na estrutura Arrange, Act e Assert:

Teste01:

CriarLoteMateriaPrima_QuantidadeInvalida_DeveLancarArgumentException (DadosServiceTestes)

- **O que verifica:** O bloqueio da criação de lotes de matéria-prima com quantidade igual a zero ou negativa.
- **Arrange:** Instanciação de um objeto *LoteMateriaPrima* com *QuantidadeKg* inválida
- **Act & Assert:** Execução assíncrona do método validando o disparo de *ArgumentException* com a mensagem "*A quantidade de matéria-prima (Kg) deve ser maior que zero.*"

Teste02:

CriarLoteMateriaPrima_PegadaCarbonoNegativa_DeveLancarArgumentException (DadosServiceTestes)

- **O que verifica:** O lançamento de exceção ao tentar cadastrar um fator de pegada de carbono negativo.
- **Arrange:** Criação do lote com *PegadaCarbonoPorKg* = -0.5.
- **Act & Assert:** *Assert.ThrowsAsync<ArgumentException>* confirmando a mensagem "O fator de Pegada de Carbono não pode ser um número negativo."

Teste03:

**ExcluirFornecedor_IdVazioOuNulo_DeveLancarArgumentException e
ExcluirVeiculo_VinVazioOuNulo_DeveLancarArgumentException (DadosServiceTestes)**

- **O que verifica:** Impedimento de chamadas de exclusão com identificadores nulos ou vazios antes do acesso à base de dados.
- **Arrange:** Passagem de strings "" e null via [InlineData].
- **Act & Assert:** Validação do disparo de ArgumentException garantindo a higienização dos parâmetros.

Teste04:

**ValidateEmailAsync_EmailComDominioIveco_DeveRetornarTrue
(EmailValidationServiceTestes)**

- **O que verifica:** A aprovação e sanitização de e-mails corporativos válidos com o domínio.
- **Arrange:** Instanciação do *EmailValidationService* com dados de entrada incluindo espaços e caracteres maiúsculos.
- **Act:** Chamada do método *ValidateEmailAsync(email)*.
- **Assert:** Asserções *Assert.True(resultado.isValid)* e verificação da mensagem de sucesso.

Teste05:

**ValidateEmailAsync_EmailSemDominioIveco_DeveRetornarFalse
(EmailValidationServiceTestes)**

- **O que verifica:** O bloqueio de e-mails externos ou com falsos domínios.
- **Arrange:** Entradas como *"usuario@gmail.com"*, *"funcionario@iveco.com.br"* e *"iveco.com@yahoo.com"*.

- **Act:** Execução do método de validação.
- **Assert:** Asserção *Assert.False(resultado.isValid)* com mensagem "Apenas e-mails com domínio @iveco.com são permitidos."

5 TÉCNICAS E RECURSOS APLICADOS

A tabela a seguir relaciona as bibliotecas, dependências do projeto e técnicas utilizadas na criação dos testes unitários:

Recurso/Ferramenta	Versão/Tipo	Aplicação nos Testes
xUnit	Framework	Gerenciamento e execução dos testes com as anotações [Fact] e [Theory] para testes simples e parametrizados.
Moq	Biblioteca Mock	Simulação das interfaces de dependência ILogger<DadosService> e IMemoryCache.
Microsoft.NET.Test.Sdk	SDK.NET	Suporte à descoberta e compilação dos testes unitários no ecossistema .NET.
Parametrização (InlineData)	Recurso xUnit	Injeção de múltiplos cenários de teste em um único método (ex: strings nulas, em branco, números negativos).

Para as validações de regras de negócio, a suíte dividiu as entradas diretamente entre cenários aceitos e rejeitados, testando as fronteiras numéricas (valores maiores que zero) e temporais (datas presentes/passadas vs. futuras). enquanto aceita valores a partir do zero, e valide o tamanho exato do VIN (aceitando apenas 17).

7 RELATO DE DEFEITOS E CORREÇÕES

Durante o desenvolvimento dos testes para o serviço *EmailValidationService*, identificou-se uma inconformidade com e-mails corporativos que continham espaços acidentais nas extremidades ou letras maiúsculas.

- **Identificação:** Identificado na execução do teste `ValidateEmailAsync_EmailComDominioIveco_DeveRetornarTrue` com a entrada "ADMIN@IVECO.COM".
- **Comportamento Observado:** A validação falhava ao tentar comparar a string bruta diretamente com a extensão do domínio, resultando em recusa indevida de e-mails válidos.
- **Comportamento Esperado:** O serviço deveria remover espaços em branco das pontas (`Trim()`) e ignorar a diferença entre maiúsculas e minúsculas antes de verificar o domínio `@iveco.com`.
- **Correção Aplicada:** Foi adicionada a sanitização do parâmetro de e-mail no método `ValidateEmailAsync()`, aplicando `.Trim().ToLowerInvariant()` antes do processamento. A alteração garantiu a aprovação de todos os testes de validação de e-mail.

8 APRENDIZADO E CONTRIBUIÇÃO PARA O PROJETO

A implementação dos testes unitários na camada de serviços garantiu que as validações essenciais do sistema Green Ledger sejam executadas com segurança no backend antes do envio de dados ao Firebase. Isso reduz chamadas desnecessárias à base de dados e evita o armazenamento de registros inconsistentes ou fora dos padrões corporativos da IVECO.

Individualmente, o desenvolvimento dessa suíte proporcionou o domínio de testes assíncronos no C#, uso avançado da biblioteca Moq para isolamento de dependências (*ILogger* e *IMemoryCache*) e a aplicação de testes parametrizados para cobertura eficiente de múltiplos cenários de erro.

